

Cadenas

Programación I

UNRN

Universidad Nacional
de Río Negro

27

Agost

0

Programación I

Clase 06: Cadenas

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Contenidos de la clase

- **Operador sizeof**
- **Arreglos**
- **Acceso (Modificaciones como R/L-Value)**
- **Identidad**
- **Recorrido**
- **Comportamiento no definido**
- **Arreglos de largo variable [VLA *Variable Length Array*]**
- **Arreglos y funciones**
- **Constantes**
- **Retorno de arreglos en funciones**
- **Cadenas (Arreglo de caracteres)**
- **Cadenas seguras**

1 Operador "sizeof" (*La Cinta Métrica de la Memoria*)

size_t sizeof(variable)

nos da el tamaño en bytes de la variable o tipo (*Aunque se escribe con paréntesis y muchas veces lo confunden con una función*)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int edad = 25;
```

```
    // Puedes medir el tipo de dato general:
```

```
    printf("Un entero (int) ocupa: %lu bytes\n", sizeof(int));
```



Un entero (int) ocupa: 4 bytes

```
    // O puedes medir una variable específica:
```

```
    printf("La variable 'edad' ocupa: %lu bytes\n", sizeof(edad));
```



La variable 'edad' ocupa: 4 bytes

```
    return 0;
```

```
}
```

%lu = l (long) u (unsigned)

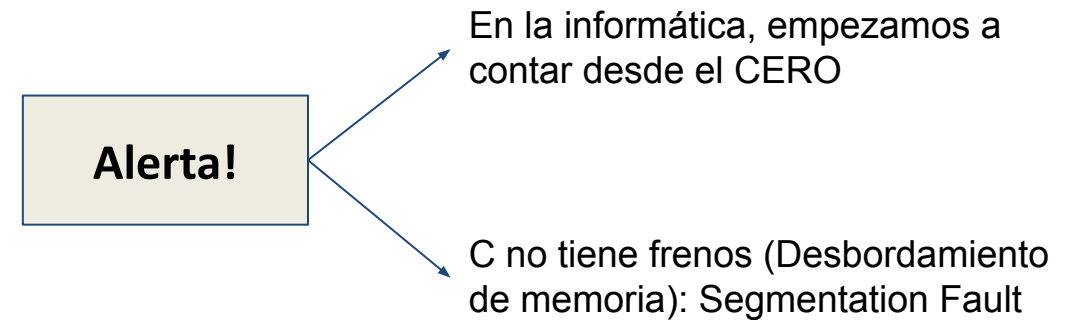
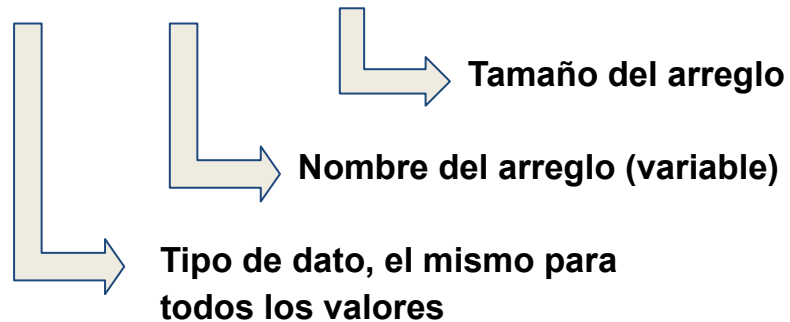
1 Operador “sizeof” (La Cinta Métrica de la Memoria)

Tipo de Dato	Tamaño típico (sizeof)	Rango de Valores (Aprox.)	¿Para qué se usa?	Ejemplo
char	1 byte	-128 a 127	Un solo carácter (letras, símbolos).	'A', '9'
int	4 bytes	-2,147,483,648 a 2,147,483,647	Números enteros sin decimales.	25, -100
float	4 bytes	1.2E-38 a 3.4E+38 (~6 decimales de precisión)	Números con decimales simples.	3.14f, 9.81f
double	8 bytes	2.3E-308 a 1.7E+308 (~15 decimales de precisión)	Números con decimales de alta precisión.	3.1415926535

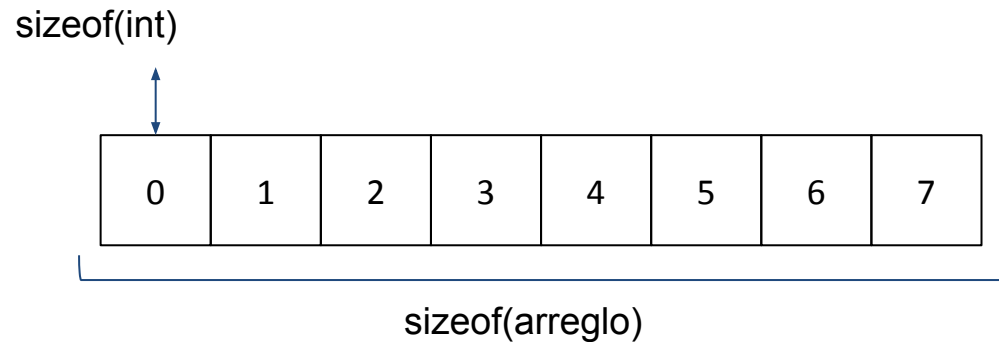
2 Arreglos

Un arreglo es una estructura que nos permite guardar **múltiples valores del mismo tipo** bajo un solo nombre, almacenados juntos en posiciones pegadas (consecutivas) dentro de la memoria RAM.

`"int arreglo[8];"`



2 Arreglos



```
#include <stdio.h>
```

```
int main() {  
    int edades[5] = {18, 21, 19, 22, 20};
```

Definimos un arreglo

```
    size_t tamano_elemento = sizeof(edades[0]);  
    size_t tamano_arreglo = sizeof(edades);
```

```
    printf("Un solo elemento (int) ocupa: %lu bytes\n", tamano_elemento);  
    printf("El arreglo completo ocupa: %lu bytes\n", tamano_arreglo);
```

```
    //aca si hago esta cuenta, deberia calcular el tamaño del arreglo, no?  
    size_t cantidad_elementos = tamano_arreglo / tamano_elemento;
```

```
    printf("\nPor lo tanto, el arreglo tiene: %lu elementos.\n", cantidad_elementos);
```

```
    return 0;  
}
```

tipo identificador[longitud];

La longitud debe ser un número!

2 Arreglos: Inicialización

```
int vacia[5] = {};
```

0	0	0	0	0
---	---	---	---	---

Me aseguro de que los valores iniciales sean '0'

```
int arreglo[5];
```

4	9	5	7	15
---	---	---	---	----

Va a contener "Valores basura" (Garbage values).

```
int arreglo[5] = {1,2,3,4,5};
```

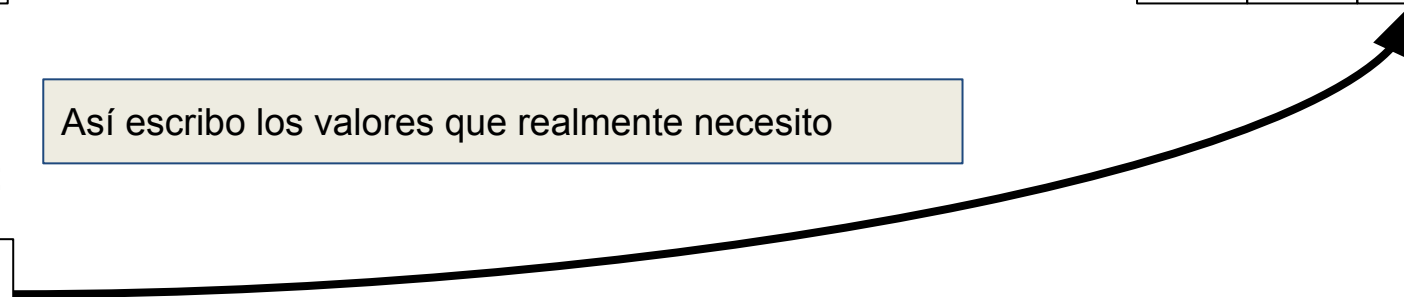
1	2	3	4	5
---	---	---	---	---

Así escribo los valores que realmente necesito

Valor

1	2	3	4	5
0	1	2	3	5

posición



3 Acceso (como R-Value): como modificador de la cadena

```
#include <stdio.h>

int main()
{
    char antes[] = "XXXXX";
    char cadena[5] = "Hola";
    char después[] = "YYYYY";
    printf("cadena: %s\n", cadena);
    printf("%c\n", cadena[-4]);
    printf("%c\n", cadena[6]);
    return 0;
}
```

Comportamiento Indefinido (Undefined Behavior)!

- Imprimir una 'X' y una 'Y' (lo que el ojo humano supone al leer el código).
- Imprimir caracteres basura invisibles o símbolos raros.
- Romperse por completo lanzando un error de protección de memoria.

4 Modificaciones (Uso como L-Value)

```
#include <stdio.h>
int main()
{
    int edades[4] = {20, 25, 22, 28};
    printf("La edad tres es: %d\n", edades[2]);
    edades[2] = 23;
    printf("La nueva edad tres es: %d\n", edades[2]);
}
```

4 Identidad

```
int arr1[5] = {1, 2, 3, 4, 5};
```

```
int arr2[5] = {10, 20, 30, 40, 50};
```

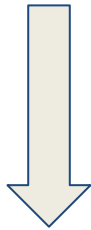
```
// La siguiente línea causará un error de compilación.
```

```
arr1 = arr2; // Error: expression is not assignable.
```

```
(se hace de otra manera "memcpy(arr1, arr2, sizeof(arr2));")
```

5 Recorrido

```
int arreglo[10] = {};  
size_t largo = sizeof(arreglo) / sizeof(arreglo[0]);  
for (size_t i = 0; i < largo; i++)  
{  
    arreglo[i] = i * i;  
}
```



0	1	4	9	16	25	36	49	64	81
0	1	2	3	4	5	6	7	8	9

aparece la variable: *size_t* :

- **Sin signo:** Solo almacena números enteros mayores o iguales a cero (no acepta valores negativos).
- **Operador *sizeof*:** Es el tipo de dato que devuelve por defecto el operador *sizeof* y funciones como *strlen*.
- **Uso común:** Se emplea en bucles que recorren arreglos o estructuras de datos para evitar desbordamientos y advertencias del compilador.

Evita errores de mantenimiento: Si mañana decides cambiar el tamaño del arreglo a `int arreglo[20]`, el cálculo automático se actualizará solo. Si pones un 10 fijo (un número "mágico"), tendrías que buscar y cambiar ese 10 en todo tu código.

Código autodocumentado: Le dice a cualquiera que lea el código que el bucle recorrerá exactamente la totalidad del arreglo, sin importar cuántos elementos tenga.

6 Comportamiento no definido

El compilador, al encontrar dicha situación, puede generar cualquier resultado.

Ejemplos:

```
int arr[5];  
int valor = arr[10];
```

```
int arreglo[10];  
for (size_t i = 0; i < 100 ; i++)  
{  
    printf("%zu/%d | ", i, arreglo[i]);  
}
```

```
int arr[5];  
arr[10] = 5;
```

6 Comportamiento no definido

En C, los límites de un arreglo existen solo en tu código fuente, no en la memoria en tiempo de ejecución. Esto es lo que sucede internamente paso a paso

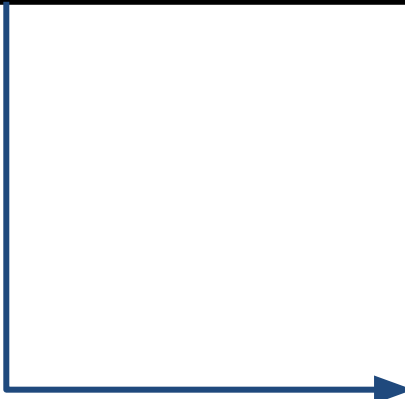
```
#include <stdio.h>

int main()
{
    int arreglo[10];
    for (size_t i = 0; i < 100 ; i++)
    {
        arreglo[i] = 0;
        printf("%zu/%d", i, arreglo[i]);
    }
}
```

6 Arreglos de largo variable [VLA]

```
int n = 5;  
// ... el usuario puede cambiar el valor de 'n'  
aquí ...  
int arreglo[n]; // El tamaño se decide cuando se  
ejecuta esta línea
```

No está recomendado, o al menos usar con cuidado,
por que puede suceder lo siguiente



```
int n; scanf("%d", &n); // El usuario ingresa  
1,000,000  
int arreglo[n]; //Desbordamiento de pila  
(Stack Overflow)
```

7 Arreglos y funciones

Regla fundamental: *“los arreglos no se copian cuando se pasan a una función”.*

```
#include <stdio.h>
```

```
// Prototipo: Recibe el arreglo y su tamaño  
void duplicarValores(int numeros[], int tamano);
```

```
int main() {  
    int misNumeros[3] = {10, 20, 30};  
  
    printf("Antes de la funcion: %d\n", misNumeros[0]); // Imprime 10  
  
    // Al llamar a la funcion, SOLO se pone el nombre del arreglo (sin corchetes)  
    duplicarValores(misNumeros, 3);  
  
    printf("Despues de la funcion: %d\n", misNumeros[0]); // ¡Imprime 20!  
  
    return 0;  
}  
// Cuerpo de la función  
void duplicarValores(int numeros[], int tamano) {  
    for(int i = 0; i < tamano; i++) {  
        numeros[i] = numeros[i] * 2; // Modifica la memoria real  
    }  
}
```

Sintaxis en la función:

- Se usan los corchetes vacíos [] en los parámetros para avisarle a C que va a recibir un arreglo.

Ejemplo de firma: void modificarEdades(int arreglo[], int tamano);

- **Error común**: Intentar pasar el arreglo al main() usando corchetes, por ejemplo: duplicarValores(misNumeros[], 3); o duplicarValores(misNumeros[3], 3);.
- **La forma correcta**: Para enviar el arreglo, solo se escribe su nombre limpio: duplicarValores(misNumeros, 3);.

7 Arreglos y funciones: Ejercicio en clase

¿Qué imprimirá este programa en la pantalla y por qué?"

```
#include <stdio.h>
```

```
void imprimirArreglo(int numeros[]) {  
    // Intentamos calcular el tamaño del arreglo en la función  
    int tamano = sizeof(numeros) / sizeof(numeros[0]);  
  
    printf("El tamaño calculado es: %d\n", tamano);  
  
    for(int i = 0; i < tamano; i++) {  
        printf("%d ", numeros[i]);  
    }  
    printf("\n");  
}  
  
int main() {  
    int misNumeros[5] = {10, 20, 30, 40, 50};  
  
    imprimirArreglo(misNumeros);  
  
    return 0;  
}
```

Que sucede?

- Cuando escribimos `int numeros[]` en los parámetros de la función, C nos hace creer que estamos recibiendo un arreglo completo. Pero en realidad, solo estamos recibiendo un **puntero** (la dirección de memoria de la primera posición).
- Cuando pasas un arreglo a una función en C, este pierde temporalmente su "medición" (`sizeof`). Al aplicar `sizeof(numeros)` dentro de la función, el operador no mide el arreglo de 5 elementos (20 bytes), sino que mide **el tamaño de la etiqueta con la dirección de memoria** (que en sistemas de 64 bits ocupa 8 bytes).
- La matemática que hace el procesador termina siendo $8 \text{ bytes} / 4 \text{ bytes} = 2$. ¡Por eso el bucle se detiene antes de tiempo! La función solo sabe dónde empieza el edificio, pero no sabe cuántos pisos tiene.

8 Constantes

En C, una **constante** es un valor que no puede ser modificado por el programa durante su ejecución

Constantes de Preprocesador (#define)

- **Sintaxis:** `#define PI 3.14159` (Nota: no lleva punto y coma al final)
- **Ventaja:** Cero consumo de memoria en tiempo de ejecución.
- **Desventaja:** Al no tener un tipo de dato explícito (int, float, etc.), es más difícil para el compilador detectar errores de tipo, y es más complicado de depurar

El Calificador const (Variables de sólo lectura)

- **Sintaxis:** `const int LIMITE_MAXIMO = 100;`
- **Ventaja:** Tiene un tipo de dato claro (int, float, char). Si intentas modificar (`LIMITE_MAXIMO = 200;`), el compilador detendrá la compilación con un error inmediatamente.

9 Retorno de arreglos en funciones

En C, **no puedes devolver un arreglo local directamente usando un `return` tradicional.**

```
#include <stdio.h>
```

```
// La función intenta devolver la dirección de un entero (un arreglo)
```

```
int* crearArregloSeguro() {
```

```
    // La palabra 'static' evita que C destruya este arreglo al hacer return
```

```
    static int numeros[3] = {10, 20, 30};
```

```
    return numeros; // Devolvemos la "dirección de memoria" del inicio
```

```
}
```

Mas adelante vamos a ver: "Memoria Dinámica" (`malloc`), como solucion para esto

```
int main() {
```

```
    // Atrapamos las llaves del edificio usando un puntero (*)
```

```
    int* miArreglo = crearArregloSeguro();
```

```
    // Ahora podemos usarlo normalmente
```

```
    printf("El primer numero es: %d\n", miArreglo[0]); // Imprime 10
```

```
    return 0;
```

```
}
```

10 Cadenas (Arreglo de caracteres) I

- El Arreglo de Caracteres: Para formar una palabra, C utiliza un arreglo (array) de variables char.
- El Problema: Como C no tiene frenos y no recuerda automáticamente el tamaño de los arreglos, ¿cómo sabe un printf cuándo dejar de imprimir letras en la pantalla? “*Toda cadena de texto en C debe terminar obligatoriamente con un carácter especial llamado carácter nulo (\0).*”
- El costo en memoria: Esto significa que si quieres guardar la palabra "Hola" (4 letras), necesitas pedirle a C una caja de 5 espacios en memoria, \0.

Comportamiento indefinido
Fallas de seguridad

```
#include <stdio.h>
```

```
int main() {  
    char palabra[6]; // 1. Creamos un arreglo con espacio suficiente (un extra para el '\0')  
    printf("Introduce una palabra (máximo 5 letras): ");  
    // Usamos %5s para evitar que el usuario sature la memoria si escribe de más  
    // No se usa el símbolo '&' antes de 'palabra' porque los arreglos ya son direcciones  
    scanf("%5s", palabra);  
    printf("La palabra guardada es: %s\n", palabra); // 3. Mostramos el resultado  
  
    return 0;  
}
```

O sea lo que necesitamos + 1

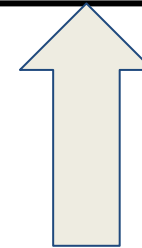
H o l a \0

10 Cadenas (Arreglo de caracteres) II Ejemplo/ejercicio

```
#include <stdio.h>
int main()
{
    char primera[] = "Hola Mundo";
    char segunda[6] = "Programacion";
    char tercera[] = "Adios Mundo\n";
    printf("primera: %s\n", primera);
    printf("segunda: %s\n", segunda);
    printf("tercera: %s\n", tercera);
    return 0;
}
```

```
#include <string.h>
int strlen ( char[] str );
```

Cuenta los caracteres hasta el \0



Cadenas seguras

10 Cadenas seguras

Todas las cadenas en funciones
deben ser seguras!!

11 libreria: stdlib

```
size_t strlen( const char* str );  
    char nombre[] = "Hola";  
    printf("Longitud: %zu\n", strlen(nombre)); // Imprime 4 (no cuenta el '\0')  
char *strcat( char *dest, const char *src );  
    char origen[] = "Programacion en C";  
    char destino[10];  
    // Copiamos como máximo 9 caracteres para dejar espacio al '\0'  
    strncpy(destino, origen, sizeof(destino) - 1);  
    destino[sizeof(destino) - 1] = '\0'; // Aseguramos el cierre  
    printf("Destino: %s\n", destino); // Imprime "Programac"
```

11 librería: stdlib

```
char* strcpy( char* dest, const char* src );  
    char saludo[20] = "Hola ";  
    char nombre[] = "Carlos";  
    // Añade "Carlos" al final de "Hola " respetando el espacio libre  
    strncat(saludo, nombre, sizeof(saludo) - strlen(saludo) - 1);  
    printf("%s\n", saludo); // Imprime "Hola Carlos"  
int strcmp( const char* lhs, const char* rhs );  
    if (strncmp(usuario, "admin", 5) == 0) { printf("Acceso correcto\n"); }
```


11 librería: stdlib

Función	Objetivo Principal	Librería	Nivel de Seguridad
<code>strlen()</code>	Medir la cantidad de letras	<code><string.h></code>	Seguro
<code>strncpy()</code>	Copiar un texto con límite	<code><string.h></code>	Seguro
<code>strncat()</code>	Unir dos textos con límite	<code><string.h></code>	Seguro
<code>strncmp()</code>	Comparar dos textos	<code><string.h></code>	Seguro
<code>strstr()</code> / <code>strchr()</code>	Buscar dentro de un texto	<code><string.h></code>	Seguro
<code>fgets()</code>	Leer texto por teclado de forma segura	<code><stdio.h></code>	Seguro

11 librería: ctype.h

```
int isalnum(int c);      int isprint(int c);
int isalpha(int c);      int ispunct(int c);
int isblank(int c);      int isspace(int c);
int iscntrl(int c);      int isupper(int c);
int isdigit(int c);      int isxdigit(int c);
int isgraph(int c);      int tolower(int c);
int islower(int c);      int toupper(int c);
```